

CS214 Written Assignment HW3

Hugo Wan, GitHub ID: HugoWan0504

Deadline: 5/25/2026

1 Boruvka's Algorithm

In this problem, I look at Boruvka's algorithm for finding a minimum spanning tree. My understanding is that the main idea is to repeatedly let every vertex, or later every component, choose its cheapest outgoing edge. After that, the algorithm contracts the selected edges and keeps working on a smaller graph.

1.1 (1)

I want to prove that for any vertex $v \in V$, the lightest edge incident to v must be included in the MST.

Let the lightest edge incident to v be

$$e = (v, u).$$

Now consider the cut

$$(\{v\}, V - \{v\}).$$

This cut separates vertex v from the rest of the graph. Since e is incident to v , it crosses this cut. Also, because e is the lightest edge incident to v , it is the lightest edge crossing this cut.

By the light-edge property of MSTs, the lightest edge crossing a cut is safe to add to an MST. Since the problem assumes all edge weights are distinct, this lightest edge is unique.

Therefore, the lightest edge incident to v must be in the MST.

The lightest incident edge of v is safe for the MST.

1.2 (2)

I want to prove that Boruvka's algorithm is correct.

From part (1), the lightest edge incident to a vertex is safe to add to the MST. In the first round of Boruvka's algorithm, every vertex chooses its lightest incident edge. So all selected edges in the first round are safe.

After that, the algorithm contracts the connected components formed by the red edges. I can think of each contracted component as a new super-vertex. In later rounds, each super-vertex chooses its lightest outgoing edge.

This is basically the same cut argument as before. If C is one current component, then I can look at the cut

$$(C, V - C).$$

The edge selected by component C is the lightest edge crossing this cut. By the light-edge property, that edge is safe to add to the MST.

So in every round, Boruvka's algorithm only adds safe edges. The contraction step does not break correctness because the contracted edges are already safe MST edges. The algorithm keeps shrinking the graph until everything is connected.

Therefore, the final selected edges form a minimum spanning tree.

Boruvka's algorithm is correct because every chosen edge is safe.

1.3 (3) Bonus

I want to describe how to contract the graph using the red edges.

The algorithm I would use is:

1. First, build the graph

$$G_{\text{red}} = (V, E_{\text{red}}).$$

This graph only contains the red edges.

2. Compute the connected components of G_{red} . Each connected component becomes one new vertex in the contracted graph G' .
3. Give every original vertex v a component label, written as $\text{comp}[v]$.
4. For every original edge $(u, v) \in E$, create an edge

$$(\text{comp}[u], \text{comp}[v]).$$

5. If $\text{comp}[u] = \text{comp}[v]$, then this edge is inside the same component. It becomes a self-loop, so I can ignore it.
6. If there are multiple edges between the same two new components, keep only the one with minimum weight.

For the duplicate edges, I can use a parallel hash table or an expected-linear-work reduce-by-key step on the component-pair endpoints. For each pair of components, I keep only the minimum-weight edge. This is like a parallel “group by minimum” step, but the important point is that the total work stays $O(m)$.

The work is $O(m)$ because the main step is scanning through all original edges. The red graph has only $O(n)$ red edges, so computing connected components on it can be done efficiently in parallel.

The parallel graph lectures discussed connected components and contraction-style algorithms, where edges are remapped after vertices are grouped together. Using a parallel connectivity algorithm, plus parallel filtering or sorting/hashing to remove self-loops and duplicate edges, the span can be made polylogarithmic.

Therefore, the contraction step can be done in

$O(m)$ work and polylogarithmic span.

1.4 (4)

I want to prove that

$$|V'| \leq \frac{|V|}{2}.$$

In one round of Boruvka's algorithm, every vertex chooses one incident edge and marks it red. This means every vertex touches at least one red edge.

Since an MST problem assumes the graph is connected, every active vertex or component has at least one incident outgoing edge until the algorithm finishes.

So in the graph

$$G_{\text{red}} = (V, E_{\text{red}}),$$

there should be no connected component with only one vertex. Every connected component has at least two vertices because every vertex is connected to another vertex by a red edge.

When the graph is contracted, each connected component becomes one vertex in G' . Since every component has at least two original vertices, the number of new vertices is at most half of the original number:

$$|V'| \leq \frac{|V|}{2}.$$

$$\boxed{|V'| \leq |V|/2}$$

1.5 (5)

Now I want to parallelize Step 1 of Boruvka's algorithm. Step 1 asks every vertex to find its lightest incident edge.

The parallel algorithm is:

1. Store the graph using adjacency lists or a CSR-style graph representation.
2. For every vertex v , look through all edges incident to v .
3. Use a parallel reduction to find the minimum-weight incident edge for each vertex.
4. Mark all of these selected edges as red.

For one vertex v , finding the minimum edge in its adjacency list takes work proportional to its degree. Across all vertices, the total size of all adjacency lists is $O(m)$ up to a constant factor.

So the work is

$$W = O(m).$$

The span for finding the minimum in one adjacency list is logarithmic in the degree of the vertex. In the worst case, the degree can be $O(n)$, so the span is

$$O(\log n).$$

This matches the work-span idea from lecture: the work counts all operations, while the span counts the longest dependency chain. A parallel reduction has logarithmic span.

Therefore, Step 1 can be done in

$$\boxed{O(m) \text{ work and } O(\log n) \text{ span.}}$$

1.6 (6)

I want to prove that the full parallel Boruvka algorithm has

$$O(m \log n)$$

work and polylogarithmic span.

From part (4), the number of vertices shrinks by at least half after each contraction. So the number of rounds is at most

$$O(\log n).$$

In each round, finding the lightest incident edges costs $O(m)$ work. Also, from the contraction result, contracting the graph costs $O(m)$ work and polylogarithmic span.

So each round costs $O(m)$ work, and there are $O(\log n)$ rounds. Therefore,

$$W = O(m) \cdot O(\log n) = O(m \log n).$$

For span, each round has polylogarithmic span. Since there are $O(\log n)$ rounds, multiplying by another logarithmic factor still gives polylogarithmic span.

$$\boxed{W = O(m \log n), \quad D = \text{polylogarithmic.}}$$

1.7 (7) Bonus

I want to prove the tighter work bound

$$O\left(\min\left(n^2, m\left(1 + \log \frac{n^2}{m}\right)\right)\right).$$

Let n_i be the number of vertices in round i . Since the number of vertices shrinks by at least half each round,

$$n_i \leq \frac{n}{2^i}.$$

In round i , the number of edges is at most m , since the graph cannot have more edges than the original graph. Also, with n_i vertices, the graph has at most $O(n_i^2)$ possible edges.

So the work in round i is bounded by

$$O(\min(m, n_i^2)).$$

Using $n_i \leq n/2^i$, this becomes

$$O\left(\min\left(m, \frac{n^2}{4^i}\right)\right).$$

So the total work is

$$\sum_i O\left(\min\left(m, \frac{n^2}{4^i}\right)\right).$$

For the early rounds, $\frac{n^2}{4^i}$ is still bigger than m , so each of those rounds costs $O(m)$. The number of these early rounds is about

$$O\left(\log \frac{n^2}{m}\right).$$

So the early rounds cost

$$O\left(m \log \frac{n^2}{m}\right).$$

After that point, the terms decrease geometrically:

$$m + \frac{m}{4} + \frac{m}{16} + \cdots = O(m).$$

Therefore,

$$W = O\left(m \left(1 + \log \frac{n^2}{m}\right)\right).$$

There is also a simple upper bound using only n_i^2 :

$$\sum_i O(n_i^2) = O\left(n^2 + \frac{n^2}{4} + \frac{n^2}{16} + \cdots\right) = O(n^2).$$

Combining the two bounds gives

$$W = O\left(\min\left(n^2, m \left(1 + \log \frac{n^2}{m}\right)\right)\right).$$

2 Fun with Treaps

A treap is a binary search tree based on keys and also a heap based on priorities. In this homework, a higher priority means the node should be closer to the root. I use the parallel tree lecture ideas here, especially the ideas of balanced trees, split, join, and set operations on trees.

2.1 Basic Treap Properties and Constructing Treaps

2.1.1 (1)

The root of the treap is the key with the highest priority.

The reason is that the treap must satisfy the heap property on priorities. Since the root has no parent, it must be the node with the largest priority.

The root is the key with maximum priority.

2.1.2 (2)

I want to explain why the height of a treap is related to the number of rounds in randomized quicksort.

In a treap, the root is the key with the highest priority. Since priorities are random, the root is like a randomly chosen key.

After choosing the root:

- all smaller keys go to the left subtree,
- all larger keys go to the right subtree.

This is exactly like quicksort choosing a random pivot:

- smaller elements go to the left side,
- larger elements go to the right side.

Then the same process happens recursively on both sides.

So the shape of a random treap is the same as the recursion tree of randomized quicksort. The height of the treap corresponds to the longest path in this recursive process. This is the same kind of quantity as the maximum number of recursive rounds in randomized quicksort.

Therefore, the height of a treap has the same probability distribution as the number of rounds in randomized quicksort.

Treap height behaves like randomized quicksort recursion depth.

2.1.3 (3)

I want to build a treap from n unsorted keys and priorities in

$$O(n \log n)$$

work and

$$O(\log^2 n)$$

span with high probability.

The algorithm is:

1. Sort the keys.
2. Find the key with the largest priority. This key becomes the root.
3. Recursively build the left subtree from keys smaller than the root.
4. Recursively build the right subtree from keys larger than the root.
5. Run the two recursive calls in parallel.

Sorting the keys can be done using a parallel sorting algorithm. From the parallel sorting idea in lecture, I can use sorting with

$$O(n \log n)$$

work and polylogarithmic span. For this homework target, I assume a standard parallel sorting routine with

$$O(\log^2 n)$$

span.

After sorting, the recursive construction is like quicksort. At each level, I find the maximum priority in each subproblem. Across one level, this costs $O(n)$ total work. Since the treap height is $O(\log n)$ with high probability, the total construction work is

$$O(n \log n).$$

For span, finding the maximum priority can be done by parallel reduction in $O(\log n)$ span per level. Since there are $O(\log n)$ levels with high probability,

$$D = O(\log n) \cdot O(\log n) = O(\log^2 n).$$

Including the parallel sorting step, the total bound is still

$W = O(n \log n), \quad D = O(\log^2 n) \text{ w.h.p.}$

2.1.4 (4) Bonus

If the keys are already sorted, then I do not need to sort them first.

In this case, the treap is like a Cartesian tree. The sorted order gives the binary search tree order, and the priorities give the heap order.

One possible algorithm is:

1. For each key, find the nearest key on the left that has larger priority.
2. For each key, find the nearest key on the right that has larger priority.
3. The parent of each node is chosen from these two candidates. If both candidates exist, I choose the one with the smaller priority among the two larger-priority candidates, since that node is the closest ancestor in the Cartesian tree.
4. Then connect all parent-child pointers in parallel.

This avoids the $O(n \log n)$ sorting cost. Using a parallel nearest-larger-element style algorithms, this can be done in linear work and polylogarithmic span.

Therefore, for sorted keys, the treap can be built in

$O(n) \text{ work and } O(\log^2 n) \text{ span w.h.p.}$
--

2.2 Joining and Splitting Treaps

2.2.1 (5)

I want to show that in the split algorithm, whenever join is called, it is always a direct join.

In split, the algorithm follows one search path in the treap. Suppose the root key is less than or equal to the split key. Then the root should stay in the left output tree, and the algorithm recursively splits the right subtree.

After the recursive call, the algorithm joins the root back with some subtrees. This join is direct because the root originally had higher priority than every node in its left and right subtrees. The recursive split only rearranges nodes from those subtrees, so none of those nodes can have higher priority than the root.

The same argument works if the root key is greater than the split key. Then the algorithm recurses on the left subtree instead.

So every join inside split is direct.

All joins inside split are direct joins.

2.2.2 (6)

The split operation only follows one path from the root down the treap.

At each node, the algorithm compares the split key with the root key. Then it recurses into either the left subtree or the right subtree, but not both.

Therefore, the cost of split is proportional to the height of the treap.

Since a random treap has height

$$O(\log n)$$

with high probability, the split cost is also

$O(\log n)$ with high probability.

2.2.3 (7) Bonus

Now I want a general join algorithm

$$\text{join}(T_L, k, T_R)$$

where k might not have the highest priority.

If T_L is empty, I can insert k with T_R . If T_R is empty, I can insert k with T_L . These are simpler cases of the same join idea.

The main idea is to choose the node with the highest priority among the roots and k .

1. If k has higher priority than the roots of both T_L and T_R , then I can directly join:

$$\text{directJoin}(T_L, k, T_R).$$

2. If the root of T_L has the highest priority, then the root of T_L should remain the root. Then I recursively join the right child of T_L with k and T_R .

3. If the root of T_R has the highest priority, then the root of T_R should remain the root. Then I recursively join T_L , k , and the left child of T_R .

In pseudocode:

```

join( $T_L, k, T_R$ ) :
    if  $T_L$  is empty: return directJoin( $\emptyset, k, T_R$ )
    if  $T_R$  is empty: return directJoin( $T_L, k, \emptyset$ )

    if  $k$  has the highest priority:
        return directJoin( $T_L, k, T_R$ )

    else if root( $T_L$ ) has the highest priority:
         $T_L$ .right = join( $T_L$ .right,  $k, T_R$ )
        return  $T_L$ 

    else:
         $T_R$ .left = join( $T_L, k, T_R$ .left)
        return  $T_R$ 

```

This keeps the binary search tree property because all keys in T_L are less than k , and all keys in T_R are greater than k . It also keeps the heap property because the highest-priority node is placed on top.

The cost is proportional to the height of the treaps, so it is

$$\boxed{O(\log n) \text{ with high probability.}}$$

2.3 Union, Intersection, and Difference on Two Treaps

2.3.1 (8)

In the union algorithm, the last line is

```
return join( $T_L, k, T_R$ ).
```

I need this join to be a direct join.

The problem is that if the algorithm always chooses k from the root of T_2 , then k might not have higher priority than nodes in T_1 . Then the final join might not be direct.

To fix this, I choose the root with the higher priority between T_1 and T_2 .

The modified idea is:

1. Compare the priorities of the roots of T_1 and T_2 .
2. Pick the root with higher priority as k .
3. Split the other treap by k .
4. Recursively union the left parts and right parts in parallel.
5. Directly join the result with k .

The modified algorithm is:

```

union( $T_1, T_2$ ) :
  if  $T_1$  is empty return  $T_2$ 
  if  $T_2$  is empty return  $T_1$ 

  if priority(root( $T_1$ )) < priority(root( $T_2$ )) :
    swap  $T_1$  and  $T_2$ 

   $k$  = key at root of  $T_1$ 
   $L_1$  = left subtree of  $T_1$ 
   $R_1$  = right subtree of  $T_1$ 
  ( $L_2, b, R_2$ ) = split( $T_2, k$ )

  in parallel:
     $T_L$  = union( $L_1, L_2$ )
     $T_R$  = union( $R_1, R_2$ )

  return join( $T_L, k, T_R$ )

```

This works because k is chosen as the higher-priority root between the two input treaps. So k has higher priority than every node that can appear in T_L and T_R . Therefore, the final join is direct.

Choose the higher-priority root, then split the other treap.

2.3.2 (9)

The same priority idea also helps for intersection and difference, but I have to be more careful than in union.

For union, the chosen key k is always included in the output. So after the recursive calls, I can always return

join(T_L, k, T_R).

For intersection and difference, k is not always included. If k is not included, then I need to combine the two resulting treaps using a helper operation join2(T_L, T_R), which joins two treaps without inserting a middle key.

Intersection. For intersection, a key should be kept only if it appears in both trees. The modified algorithm is:

```

intersection( $T_1, T_2$ ) :
  if  $T_1$  is empty or  $T_2$  is empty return empty

  choose the root with higher priority between  $T_1$  and  $T_2$ 

```

```

if root( $T_1$ ) has higher priority:
     $k$  = root key of  $T_1$ 
     $(L_2, b, R_2) = \text{split}(T_2, k)$ 
    in parallel:
         $T_L = \text{intersection}(T_1.\text{left}, L_2)$ 
         $T_R = \text{intersection}(T_1.\text{right}, R_2)$ 
    if  $b$  is true return  $\text{join}(T_L, k, T_R)$ 
    else return  $\text{join2}(T_L, T_R)$ 

else:
     $k$  = root key of  $T_2$ 
     $(L_1, b, R_1) = \text{split}(T_1, k)$ 
    in parallel:
         $T_L = \text{intersection}(L_1, T_2.\text{left})$ 
         $T_R = \text{intersection}(R_1, T_2.\text{right})$ 
    if  $b$  is true return  $\text{join}(T_L, k, T_R)$ 
    else return  $\text{join2}(T_L, T_R)$ 

```

Here, b tells whether the key k appeared in the other tree. If b is true, then k appears in both trees, so I keep it. Otherwise, I remove it.

Difference. For difference $T_1 - T_2$, I keep keys that appear in T_1 but not in T_2 .
The modified algorithm is:

```

difference( $T_1, T_2$ ) :
    if  $T_1$  is empty return empty
    if  $T_2$  is empty return  $T_1$ 

    choose the root with higher priority between  $T_1$  and  $T_2$ 

    if root( $T_1$ ) has higher priority:
         $k$  = root key of  $T_1$ 
         $(L_2, b, R_2) = \text{split}(T_2, k)$ 
        in parallel:
             $T_L = \text{difference}(T_1.\text{left}, L_2)$ 
             $T_R = \text{difference}(T_1.\text{right}, R_2)$ 
        if  $b$  is false return  $\text{join}(T_L, k, T_R)$ 
        else return  $\text{join2}(T_L, T_R)$ 

    else:
         $k$  = root key of  $T_2$ 
         $(L_1, b, R_1) = \text{split}(T_1, k)$ 
        in parallel:
             $T_L = \text{difference}(L_1, T_2.\text{left})$ 
             $T_R = \text{difference}(R_1, T_2.\text{right})$ 
        return  $\text{join2}(T_L, T_R)$ 

```

In the second case for difference, k comes from T_2 , so it should not be included in $T_1 - T_2$. That is why I return $\text{join2}(T_L, T_R)$.

References

- Professor Yihan Sun, CS214 Assignment 3 Handout.
- Professor Yihan Sun, *Implementing Parallel Algorithms*, CS214 Lecture Slides. Used for the fork-join model, work, span, and parallel-for/reduction analysis.
- Professor Yihan Sun, *Parallel Sequence Algorithms*, CS214 Lecture Slides. Used for parallel reduction/scan-style thinking and work-span examples.
- Professor Yihan Sun, *Parallel Sorting*, CS214 Lecture Slides. Used for sorting, parallel divide-and-conquer, and parallel filtering/packing ideas.
- Professor Yihan Sun, *Parallel Graph Algorithms*, CS214 Lecture Slides. Used for graph definitions, graph representation, and the idea of processing edges in parallel.
- Professor Yihan Sun, *Parallel Graph Algorithms: Parallel Connectivity*, CS214 Lecture Slides. Used for connected components, contraction, and polylogarithmic-span graph algorithms.
- Professor Yihan Sun, *Parallel Trees*, CS214 Lecture Slides. Used for balanced tree background and the relationship between tree height and efficient operations.
- Professor Yihan Sun, *Parallel Balanced Binary Trees*, CS214 Lecture Slides. Used for join, split, union, intersection, and difference ideas on trees.
- OpenAI. ChatGPT (GPT-5.5 Thinking). Used for helping draft explanations, checking clarity, and formatting the report in LaTeX.
https://chatgpt.com/s/t_6a0bbefee1708191b9e5bdfa94fda133